

FreelanceControl

Informe final — Capítulos de cierre

Implementación · Pruebas · Conclusiones

Trabajo Final de Grado — Ingeniería en Sistemas de Información
Versión al 31/05/2026

Capítulo: Implementación

Introducción

El presente capítulo describe las decisiones técnicas adoptadas durante la construcción del prototipo funcional de FreelanceControl. La exposición se organiza siguiendo los seis objetivos específicos del proyecto, de modo que cada decisión de implementación pueda trazarse hasta el objetivo que la justifica.

El sistema se construyó sobre una arquitectura de tres capas contenedorizadas con Docker Compose: una capa de presentación implementada en React, una capa de lógica de negocio y API REST implementada en FastAPI sobre Python 3.11, y una capa de persistencia en PostgreSQL 15. La comunicación entre el frontend y la base de datos es siempre mediada por la API; no existe acceso directo, lo que centraliza la validación, la autorización y las reglas de negocio en un único punto.

Objetivo 1 — Registro y gestión de ingresos, gastos y facturas

El núcleo transaccional se implementó como un conjunto de recursos REST con operaciones CRUD completas, persistidos mediante el ORM SQLAlchemy 2.0 sobre PostgreSQL. El esquema de la base de datos se gestiona con migraciones versionadas de Alembic (cinco revisiones al cierre del prototipo), lo que permite reproducir la estructura de la base en cualquier entorno de forma determinística.

La autenticación se resolvió con JSON Web Tokens (JWT) de vigencia configurable —siete días por defecto— firmados con el algoritmo HS256. Las contraseñas se almacenan exclusivamente como hash bcrypt, nunca en texto plano. Dos decisiones de diseño refuerzan la seguridad: en primer lugar, el identificador del usuario se extrae siempre del token y nunca del cuerpo de la solicitud, lo que impide el acceso cruzado entre cuentas; en segundo lugar, el endpoint de inicio de sesión devuelve el mismo mensaje de error tanto si el correo no existe como si la contraseña es incorrecta, evitando la enumeración de usuarios registrados.

Las facturas incorporan una máquina de estados (PENDIENTE → PAGADA / VENCIDA) con una regla de inmutabilidad: una factura en estado PAGADA no puede editarse ni eliminarse, devolviendo el código HTTP 409. Esta restricción protege la integridad del historial contable frente a modificaciones accidentales.

Objetivo 2 — Clasificador automático de gastos con PLN local

El clasificador de gastos constituye el componente de aprendizaje automático central del sistema. Se entrenó un modelo base de tipo Support Vector Machine (SVM) lineal sobre una vectorización TF-IDF de las descripciones, utilizando un conjunto de 600 ejemplos etiquetados distribuidos en doce categorías. La elección entre SVM y Naive Bayes se realiza de forma automática según el volumen de datos disponible.

La decisión más relevante de este módulo es que **toda la clasificación se ejecuta de forma local**: la descripción del gasto nunca abandona la infraestructura del sistema. Esta característica

responde directamente al objetivo de soberanía de datos y diferencia al prototipo de las soluciones que delegan la clasificación en servicios de terceros.

El flujo de resolución de una clasificación sigue tres pasos. Primero, el sistema consulta si el usuario corrigió previamente esa misma descripción; de existir una corrección registrada, se devuelve la categoría con confianza máxima sin invocar el modelo, aplicando una normalización canónica (descomposición NFKD, eliminación de tildes, colapso de espacios y conversión a minúsculas) que tolera variantes tipográficas. Segundo, si no hay corrección previa, se invoca el modelo SVM. Tercero, si la confianza de la predicción resulta inferior al umbral de 0,30, el sistema asigna la categoría "Otros" y marca el gasto como sujeto a revisión manual, en lugar de forzar una categoría poco fiable.

El sistema admite además el reentrenamiento del modelo. Cuando un usuario acumula veinte o más gastos propios, se entrena un modelo personalizado que combina el dataset base con sus ejemplos reales. Las correcciones manuales se persisten y alimentan los reentrenamientos posteriores, que se ejecutan en segundo plano para no bloquear la respuesta al usuario.

Objetivo 3 — Proyección de ingresos con series temporales

La proyección de ingresos se implementó con la biblioteca Prophet, que modela series temporales capturando tendencia y estacionalidad. Cada proyección mensual se acompaña de un intervalo de confianza con límite inferior y superior, que se almacena en la base de datos junto con el valor estimado.

Se incorporó una estrategia de arranque en frío: cuando el usuario dispone de menos de diez ingresos históricos —volumen insuficiente para que Prophet sea fiable— el sistema recurre automáticamente a una media móvil simple. Cada generación de proyecciones reemplaza las anteriores, evitando la acumulación de versiones obsoletas.

Objetivo 4 — Auditoría automatizada

El módulo de auditoría se ejecuta bajo demanda y aplica cuatro detectores en secuencia. El primero identifica gastos duplicados, definidos como aquellos con idéntico monto, categoría y descripción normalizada dentro de una ventana de tres días. El segundo detecta anomalías estadísticas mediante el cálculo del puntaje z de cada gasto respecto de la media de su categoría, marcando como anómalos los que superan dos desviaciones estándar; este detector solo actúa sobre categorías con al menos cinco gastos, condición necesaria para que la desviación estándar sea estadísticamente significativa. El tercero detecta discrepancias de facturación, señalando las facturas en estado pendiente cuya fecha de vencimiento ya transcurrió. El cuarto verifica la ausencia del pago mensual del Monotributo.

Antes de regenerar las alertas, el módulo elimina las no resueltas previas para evitar duplicaciones, mientras conserva las marcadas como resueltas a modo de historial auditable.

Objetivo 5 — Reportes consolidados e IA generativa

La generación de reportes se resolvió con ReportLab de forma programática, sin depender de un motor de renderizado HTML. El reporte mensual en PDF consolida seis secciones: encabezado, resumen ejecutivo con comparativa contra el mes anterior, estado fiscal del Monotributo,

distribución de gastos por categoría, facturación del período y resultados de la auditoría. Todos los montos se expresan en formato argentino —separador de miles con punto y decimales con coma— y el documento incluye un descargo de responsabilidad que aclara que no reemplaza el asesoramiento de un contador matriculado.

El asistente de IA generativa se integró mediante la API de Groq con el modelo LLaMA 3.3. Su uso se restringe deliberadamente a la generación de texto narrativo sobre **datos numéricos agregados**: resúmenes financieros y recomendaciones. Para cada función existe un mecanismo de respaldo local determinístico que se activa cuando el servicio externo no está disponible, garantizando que el sistema nunca quede inoperante por una dependencia externa.

Objetivo 6 — Soberanía de datos y privacidad

La soberanía de datos no se trató como una característica aislada sino como un principio transversal. La clasificación de gastos corre íntegramente en local; la detección de columnas en la importación de extractos bancarios se realiza con heurísticas locales; y el asistente de IA recibe únicamente agregados numéricos, nunca descripciones individuales, nombres de clientes ni datos identificables. De este modo, la información sensible del usuario permanece bajo su control en todos los flujos del sistema.

Capítulo: Pruebas

Estrategia de verificación

La calidad del prototipo se verificó en tres niveles complementarios: pruebas automatizadas unitarias y de integración, pruebas de humo extremo a extremo sobre la base de datos real, y evaluación cuantitativa del modelo de aprendizaje automático. Esta combinación cubre tanto la corrección funcional de los endpoints como el comportamiento del sistema en condiciones realistas y el desempeño del componente de inteligencia artificial.

Pruebas automatizadas

Se desarrolló una suite de 97 pruebas automatizadas distribuidas en once módulos, ejecutadas con pytest sobre una base de datos SQLite en memoria que se crea y destruye en cada corrida. Esta configuración garantiza que las pruebas sean rápidas, deterministas y aisladas entre sí. Los módulos cubren la autenticación, el CRUD de ingresos, gastos y facturas, la auditoría, el clasificador, la importación, el módulo de Monotributo, el reentrenamiento del modelo y la generación de reportes. Al cierre del prototipo, la totalidad de las pruebas se ejecuta de forma satisfactoria.

Pruebas de humo extremo a extremo

Dado que las pruebas automatizadas utilizan SQLite, se complementaron con una batería de pruebas de humo ejecutadas directamente contra el contenedor de PostgreSQL, replicando el entorno de producción. Estas pruebas validaron el cruce predictivo entre Prophet y el semáforo del Monotributo, el ciclo completo del clasificador (clasificar, corregir, reentrenar y reclasificar), la importación de extractos de tres bancos argentinos con formatos heterogéneos, y la generación del reporte PDF con un conjunto de datos denso.

Este nivel de prueba resultó especialmente valioso porque expuso defectos que las pruebas sobre SQLite no podían detectar. El más significativo fue una incompatibilidad en los valores de un tipo enumerado de PostgreSQL: la migración agregaba el valor en minúsculas mientras que SQLAlchemy lo emitía en mayúsculas, provocando un error al ejecutar la auditoría. El defecto solo se manifestaba sobre PostgreSQL real, lo que confirma la importancia de verificar el sistema sobre el motor de base de datos definitivo. Otros hallazgos incluyeron la falta de detección del separador de punto y coma en archivos CSV de ciertos bancos y una inconsistencia en el formato de los montos dentro de las descripciones de alertas, ambos corregidos y verificados.

Evaluación cuantitativa del clasificador

El modelo base se evaluó mediante validación cruzada de cinco particiones sobre los 600 ejemplos etiquetados. La exactitud global alcanzada fue del **76,00%** sobre doce categorías, un resultado adecuado para la tarea considerando que se trata de clasificación de texto corto en español con vocabulario heterogéneo y que el umbral de confianza deriva los casos dudosos a revisión manual.

El análisis por categoría revela un desempeño dispar pero coherente con la naturaleza del problema. Las categorías con vocabulario distintivo obtuvieron los mejores resultados: Monotributo ($F1 = 0,889$), Transporte ($F1 = 0,880$) e Impuestos ($F1 = 0,860$). En el extremo opuesto, la categoría "Otros" ($F1 = 0,489$) y "Servicios" ($F1 = 0,592$) presentaron el desempeño más bajo, un comportamiento esperable dado que ambas funcionan como categorías semánticamente amplias que se solapan con varias otras. La matriz de confusión confirma este diagnóstico: las confusiones se concentran en la columna "Otros", lo que indica que el modelo, ante la duda, tiende a la categoría genérica antes que a una incorrecta específica —precisamente el comportamiento conservador que el diseño busca.

Las métricas detalladas, la matriz de confusión completa y los gráficos correspondientes se incluyen como anexos de este informe y son reproducibles mediante los scripts ``evaluar_modelo.py`` y ``docs/gen_graficos_metricas.py``.

Capítulo: Conclusiones

Cumplimiento de los objetivos

El prototipo desarrollado cumple con los seis objetivos específicos planteados. El núcleo transaccional con autenticación y persistencia relacional quedó plenamente operativo (objetivo 1); el clasificador automático de gastos basado en procesamiento de lenguaje natural local sugiere categorías con una exactitud del 76% y mejora con las correcciones del usuario (objetivo 2); el módulo de proyección con Prophet estima la facturación futura con intervalos de confianza y estrategia de arranque en frío (objetivo 3); la auditoría automatizada detecta cuatro clases de inconsistencias (objetivo 4); el sistema genera reportes PDF consolidados e integra un asistente de IA con respaldo local (objetivo 5); y la soberanía de datos se garantizó como principio transversal en todos los flujos que manipulan información sensible (objetivo 6).

La totalidad de las trece historias de usuario formuladas en el backlog del producto fue implementada y verificada, tanto mediante pruebas automatizadas como mediante pruebas de humo sobre el entorno de producción.

Limitaciones

El prototipo presenta limitaciones que conviene reconocer. El clasificador alcanza una exactitud del 76%, lo que implica que aproximadamente uno de cada cuatro gastos requiere corrección manual; si bien el mecanismo de aprendizaje incremental mitiga este efecto con el uso, la precisión inicial depende de la representatividad del dataset base. El módulo de Monotributo contempla únicamente la actividad de servicios, sin cubrir el régimen de venta de bienes. La verificación del pago de la cuota se infiere de la existencia de un gasto categorizado como Monotributo, sin integración con los sistemas del organismo recaudador. Por último, las pruebas automatizadas se ejecutan sobre SQLite y no sobre PostgreSQL, brecha que se compensó parcialmente con las pruebas de humo pero que constituye una deuda técnica.

Trabajos futuros

A partir de las limitaciones identificadas se proponen varias líneas de trabajo futuro. En el plano del modelo de aprendizaje, la ampliación y el balanceo del dataset base, especialmente en las categorías "Otros" y "Servicios", permitirían elevar la exactitud global. En el plano fiscal, la incorporación del régimen de venta de bienes y la integración con los servicios de facturación electrónica del organismo recaudador aportarían precisión al cálculo del estado del Monotributo. En el plano de la ingeniería, la migración de las pruebas de integración a PostgreSQL, la incorporación de un control de salud de la base de datos en el orquestador de contenedores y la implementación de un registro estructurado de eventos fortalecerían la robustez operativa. Finalmente, la gestión de clientes como entidad propia y la notificación proactiva de alertas por correo o canales push ampliarían el alcance funcional del sistema.

Reflexión final

FreelanceControl demuestra que es posible construir una herramienta de gestión financiera y fiscal para trabajadores independientes que combine técnicas de aprendizaje automático con un

compromiso estricto con la privacidad del usuario. La decisión de ejecutar la clasificación de forma local, lejos de ser una restricción, se reveló como una característica diferenciadora coherente con las preocupaciones contemporáneas sobre el manejo de datos personales. El sistema sienta una base sólida y extensible sobre la cual desarrollar las líneas futuras planteadas.

Anexo — Métricas del clasificador

Resultados de la evaluación del modelo base mediante validación cruzada de cinco particiones sobre 600 ejemplos etiquetados. Exactitud global: 76,00%.

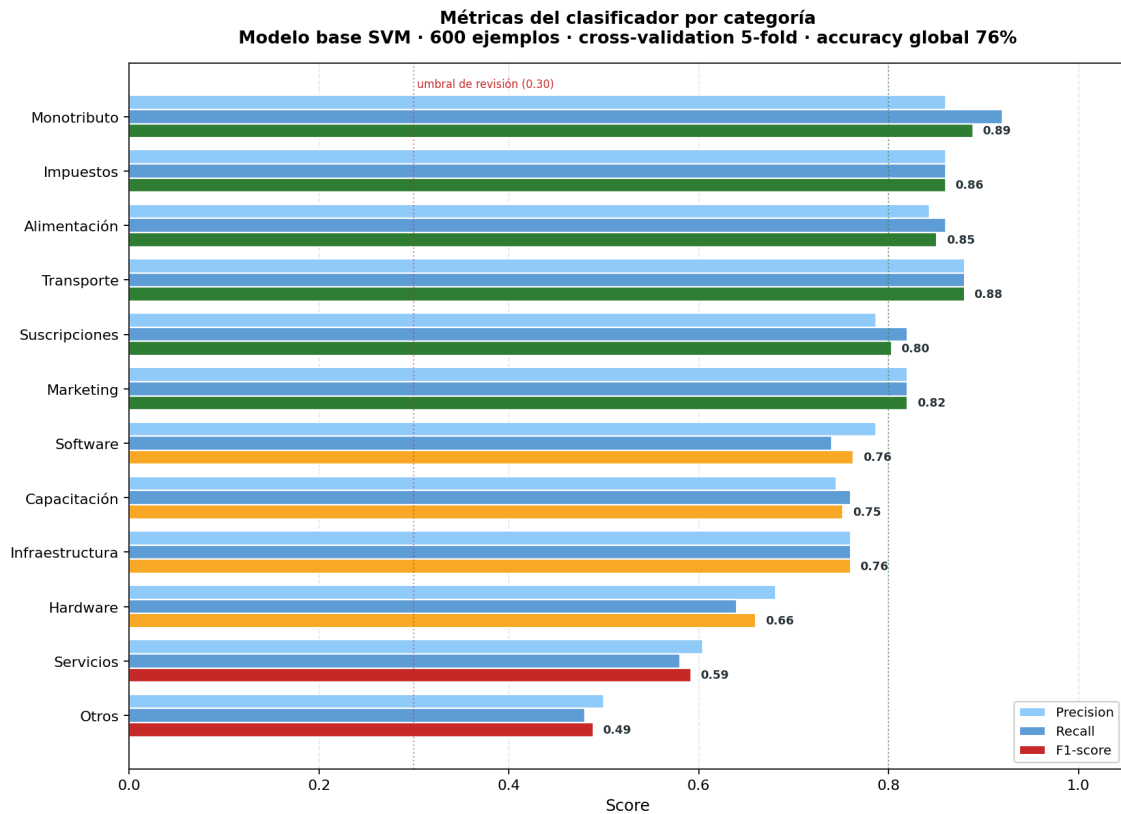


Figura 1. Precision, recall y F1-score por categoría.

Anexo — Matriz de confusión

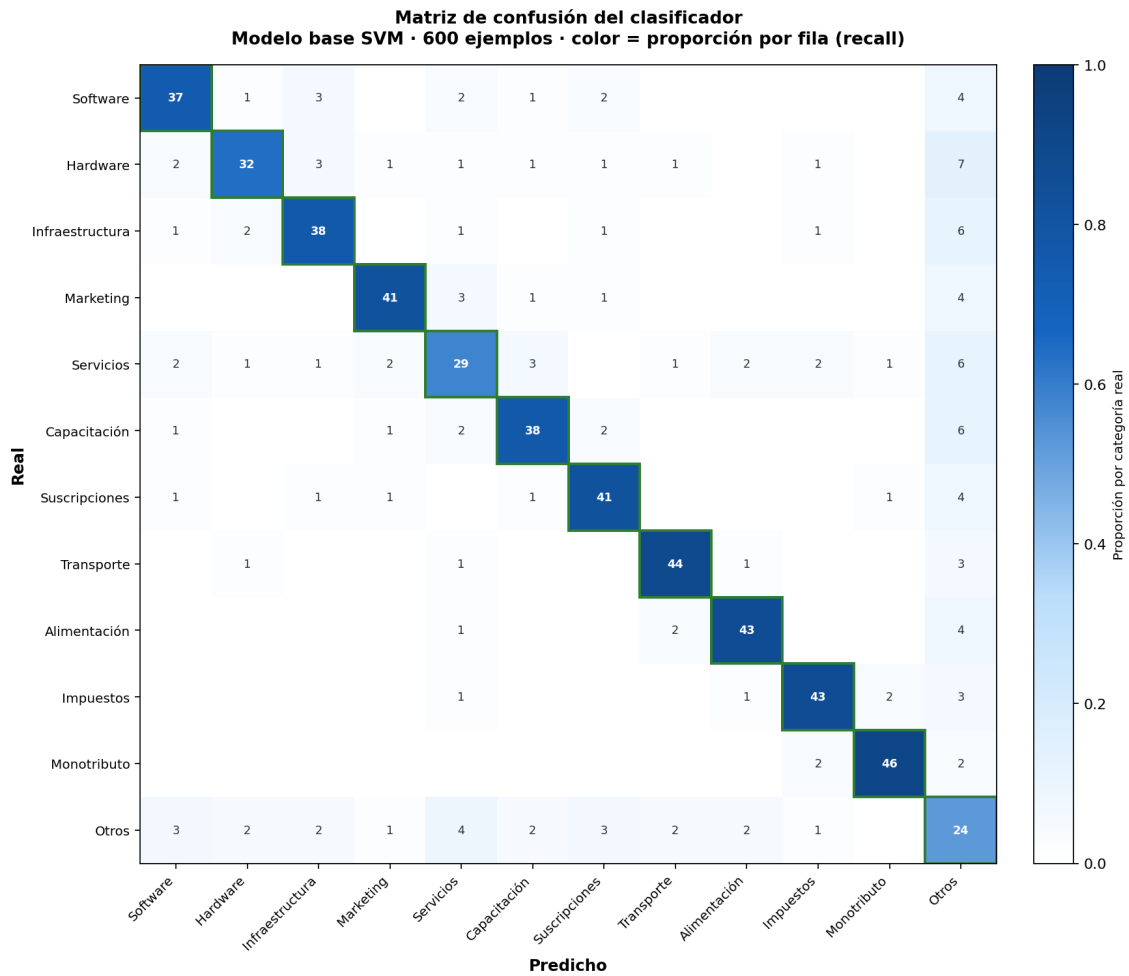


Figura 2. Matriz de confusión (filas = categoría real, columnas = predicción). El color indica la proporción por fila; la diagonal verde marca los aciertos.